STAMPEDE: Doubly-Parallel Particle MCMC via Twisted Particle Filters and DEER

Anonymous Author(s)
Affiliation
Address
email

Abstract

Sequential Monte Carlo (SMC) is the standard tool for inference in nonlinear state-space models, but two structural features of particle filters frustrate parallel hardware: forward-recursive resampling couples all particles into a single sequential update, and the stochasticity of the resulting marginal-likelihood estimator obstructs gradient-based parameter inference. We show that both obstacles dissolve under a single change of perspective. A twisted particle filter, by absorbing future-conditional information into the proposal, decouples into N *independent* per-particle nonlinear recurrences over time; each recurrence then admits a DEER-style Jacobian-linearized associative scan, yielding an $O(\log T)$ parallel-depth particle smoother whose output is bit-equal to the sequential reference. Building on this primitive, we introduce **STAMPEDE**, a doubly-parallel particle MCMC algorithm: the smoother evaluates a frozen-randomness, deterministic pseudo-marginal likelihood, and DEER is applied a second time across the chain index of a MALA sampler. Across linear-Gaussian, stochastic-volatility, Lorenz-63, and Lorenz-96 benchmarks we measure up to $200\times$ wall-clock speedup of the smoother and 2.5–$2.6\times$ end-to-end speedup of STAMPEDE over standard bootstrap particle MCMC, while *matching or exceeding* the posterior fidelity of sequential twisted PMCMC—in one case removing a 4.4σ bias that the bootstrap baseline incurs. The same smoother drops into particle EM on a Poisson linear dynamical system as a $7.2\times$-faster E-step, demonstrating that the per-particle factorization is broadly useful beyond MCMC.

1 Introduction

Particle filters (PFs), also known as sequential Monte Carlo (SMC) methods, approximate filtering and smoothing distributions in state-space models (SSMs) and have seen enduring use across robotics, neuroscience, finance, and signal processing. Their broader adoption has long been limited by computational cost, and—more fundamentally—by the prevailing view that these methods are inherently *sequential* along the temporal dimension. Recent work has begun to overturn that view: many sequential algorithms, including Markov chain Monte Carlo (MCMC) samplers, can be recast as parallel-in-time procedures. When the updates can be written as nonlinear recurrences over the full trajectory, they can be accelerated by iteratively linearizing the recurrence and evaluating the resulting linear dynamical system in parallel across time, with $O(\log T)$ depth on parallel hardware (**??**).

Standard particle filters resist this treatment for a simple structural reason: resampling. At every time step, the weights of all N particles are renormalized and a new ancestor is drawn for each, coupling the particles into a single ND-dimensional nonlinear recursion whose Jacobian is dense across particle indices. Iterative linearization is then expensive in both memory and depth, and the resulting linear system no longer decouples across particles. Consequently, although parallel-in-time algorithms have

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

been developed for Kalman smoothing (**?**), variational state-space inference (**?**), and certain MCMC samplers (**?**), $O(\log T)$ parallelism has remained out of reach for particle-based inference without sacrificing the very mechanism—resampling—that makes SMC statistically efficient.

Our starting point is the observation that twisted particle filters (**???**) sidestep this obstacle. Twisted proposals absorb future-conditional information into the one-step transition; under the optimal twist, the incremental importance weights become constant along each trajectory and resampling is unnecessary. A well-chosen twist therefore *recovers* the factorized structure that naive PFs destroy: each of the N particle trajectories evolves as an independent nonlinear recurrence over time, with a Jacobian that is block-diagonal across the particle index. This is precisely the structure that parallel-in-time solvers exploit. We show that applying a DEER-style associative scan to the Jacobian-linearized recurrence—independently for each particle, all N in parallel on a single device—yields a sequence-parallel particle smoother with $O(\log T)$ parallel depth, while preserving the unbiased likelihood estimator and consistent smoothing marginals of the underlying twisted filter.

This reformulation also changes what is possible for parameter inference. Particle MCMC (**?**), and in particular pseudo-marginal MALA (**?**), couples SMC likelihood estimation with gradient-based sampling over model parameters, but standard implementations are doubly expensive: each chain step requires a full sequential particle-filter sweep, and the stochasticity of the estimator complicates the use of gradients. *Freezing* the particle-filter randomness across proposals—as in correlated PMCMC (**?**)—turns the surrogate log-likelihood into a deterministic, differentiable function of the parameters, and the twisted-DEER smoother makes each evaluation parallel in time. We build on this synthesis to introduce **STAMPEDE**, a doubly-parallel particle MCMC algorithm that parallelizes simultaneously across time (DEER within each likelihood evaluation) and across Markov chain steps (DEER across proposed parameters), reducing wall-clock cost from $O(TK)$ to $O(\log T \cdot \log K_{\mathrm{blk}})$ parallel depth per block of K_{blk} chain steps.

Contributions.

- **A sequence-parallel twisted particle smoother.** We identify the per-particle factorization induced by twisted proposals as the structure DEER requires, and use it to build an $O(\log T)$-depth particle smoother whose output is bit-equal to the sequential reference. We characterize when DEER's diagonal Hutchinson approximation suffices, and show that a full (D, D)-block Jacobian variant recovers convergence on coupled latent dynamics with $D \lesssim 64$.
- **STAMPEDE, doubly-parallel particle MCMC.** We combine the smoother with a frozen-randomness MALA chain over the parameters and apply DEER a second time across the chain index. Quasi-ELK damping (**?**) stabilizes the inner Newton step on saturating nonlinearities, and an amortized neural twist generalizes look-ahead to the entire MCMC parameter sweep.
- **Empirical validation.** On a suite of state-space models we measure up to $200\times$ wall-clock speedup of the smoother (linear-Gaussian, $T = 16384$), $19\times$ speedup on stochastic volatility, and $2.8\times$ on Lorenz-63 with a learned full-covariance twist that matches an EKF smoother and beats FFBSi on RMSE. End-to-end, STAMPEDE delivers 2.5–$2.6\times$ speedup over standard bootstrap PMCMC while matching or exceeding sequential twisted PMCMC on every posterior diagnostic; on stochastic volatility it removes a 4.4σ bias on the persistence parameter that the bootstrap baseline incurs. As a non-MCMC application, the smoother drops in as a $7.2\times$-faster particle-EM E-step on a Poisson linear dynamical system.

2 Background

We collect the four pieces our method composes: state-space models, sequential Monte Carlo, twisted particle filters, and DEER. A reader familiar with particle MCMC may skim everything except the connection between twisting and per-particle factorization (Section 3), which is the technical pivot of this paper.

State-space models. A discrete-time state-space model has latent states $x_{1:T} = (x_1, \ldots, x_T)$ with $x_t \in \mathbb{R}^D$, observations $y_{1:T}$, and parameters $\theta \in \Theta$, specified by an initial distribution $\mu_\theta(x_1)$, a Markov transition $f_\theta(x_t \mid x_{t-1})$, and an observation likelihood $g_\theta(y_t \mid x_t)$ (**?**). The two quantities

we want are the smoothing distribution $p_\theta(x_{1:T} \mid y_{1:T})$ and the marginal likelihood

$$p_\theta(y_{1:T}) = \int \mu_\theta(x_1) \prod_{t=2}^{T} f_\theta(x_t \mid x_{t-1}) \prod_{t=1}^{T} g_\theta(y_t \mid x_t)\, dx_{1:T},$$

both intractable outside a few special cases (linear-Gaussian, discrete-state).

Sequential Monte Carlo. SMC approximates $p_\theta(x_{1:T} \mid y_{1:T})$ with a weighted ensemble $\{(x^i_{1:T}, w^i_T)\}_{i=1}^N$, propagated forward in time. The bootstrap particle filter samples $x^i_t \sim f_\theta(\cdot \mid x^{a^i_{t-1}}_{t-1})$ from the prior, weights by $w^i_t \propto g_\theta(y_t \mid x^i_t)$, and resamples ancestors a^i_{t-1} at every step to keep the weight distribution from collapsing (**??**). Resampling is what couples all N particles into a single joint update; without it, weight degeneracy ruins the estimator. In return, SMC delivers an *unbiased* marginal-likelihood estimator $\hat{p}_\theta(y_{1:T}) = \prod_{t=1}^T \frac{1}{N} \sum_i w^i_t$ (**?**), which is what makes particle MCMC valid. The cost is that the recursion is ancestor-coupled and inherently sequential.

Twisted particle filters. Twisting (**???**) replaces the transition $f_\theta(x_t \mid x_{t-1})$ with a look-ahead proposal

$$q^\psi_t(x_t \mid x_{t-1}) \;\propto\; f_\theta(x_t \mid x_{t-1})\, \psi_t(x_t),$$

where the twist $\psi_t : \mathbb{R}^D \to \mathbb{R}_{>0}$ approximates the conditional evidence $\psi^\star_t(x_t) = p_\theta(y_{t:T} \mid x_t)$. Under the *optimal* twist, the incremental importance weights collapse to a constant along every trajectory, resampling is unnecessary, and each particle evolves independently under q^ψ_t. In practice ψ_t is learned or approximated—via Gaussian look-aheads, iterated auxiliary PFs (**?**), controlled SMC (**?**), or backward-information-filter (BIF) targets—and the resulting weights are not constant but typically far better behaved than the bootstrap weights. Crucially, the per-particle recurrence structure is preserved up to (at most) a much weaker residual resampling; this structure is the lever our method pulls.

Parallel-in-time nonlinear recurrences (DEER). Given a nonlinear recurrence $h_t = F(h_{t-1}, u_t)$ for $t = 1, \ldots, T$, DEER (**?**) solves for the full trajectory $h_{1:T}$ with $O(\log T)$ parallel depth via Newton-like fixed-point iteration. At outer iteration k, DEER linearizes F about the current guess $h^{(k)}_{1:T}$,

$$h^{(k+1)}_t = J^{(k)}_t h^{(k+1)}_{t-1} + b^{(k)}_t, \qquad J^{(k)}_t = \tfrac{\partial F}{\partial h}\big|_{(h^{(k)}_{t-1}, u_t)},$$

and solves the resulting *linear* recurrence with a parallel prefix (associative) scan (**??**) of depth $O(\log T)$. Convergence is local-quadratic in the regime of standard Newton, and a small number of outer iterations typically suffices, giving total depth $O(\text{iters} \cdot \log T)$ (**??**). The quasi-ELK extension (**?**) adds Kalman-style shrinkage to the Newton step for ill-conditioned Jacobians.

Particle MCMC and pseudo-marginal MALA. Particle MCMC (**?**) targets $p(\theta \mid y_{1:T}) \propto p(\theta) p_\theta(y_{1:T})$ by substituting the unbiased SMC estimator $\hat{p}_\theta(y_{1:T})$ for the intractable marginal likelihood in the Metropolis–Hastings acceptance ratio, yielding a valid pseudo-marginal sampler (**?**). When $\log \hat{p}_\theta$ is differentiable in θ, Metropolis-adjusted Langevin (MALA) variants exploit gradient information to scale to higher-dimensional θ (**?**). The catch is that $\hat{p}_\theta$ is stochastic in the SMC randomness $\mathbf{u}$, so naive gradient estimates have high variance. *Freezing* $\mathbf{u}$ across proposals (**??**) turns the surrogate log-likelihood $\ell(\theta; \mathbf{u}) := \log \hat{p}_\theta(y_{1:T}; \mathbf{u})$ into a deterministic function of θ, differentiable via reverse-mode autodiff. STAMPEDE samples this surrogate; refreshing $\mathbf{u}$ between blocks of MALA steps preserves the correct stationary distribution.

Related parallelism in SMC and MCMC. Parallel-in-time inference has a long history for *linear-Gaussian* state-space models, starting with the parallel Kalman smoother of **?** and extended to parallel variational inference for nonlinear SSMs (**?**). For SMC, prior work has parallelized *across particles* (island filters (**?**)) or *across data* (divide-and-conquer SMC (**?**)), but not *across time within a single particle filter sweep*: resampling has been the obstacle. The closest precedents to our approach are ELK (**?**) and quasi-ELK (**?**), which parallelize Newton-like fixed-point iterations—including MCMC samplers—by treating them as nonlinear recurrences. We adopt this lens for both halves of particle MCMC: DEER over the chain of MALA proposals (following **?**), and DEER *inside* each likelihood evaluation, made possible by the per-particle factorization that twisted proposals provide.

3 Sequence-parallel twisted particle smoother

3.1 Per-particle nonlinear recurrences

The technical pivot of our method is that, under a twisted proposal, each particle's trajectory is a self-contained nonlinear recurrence in the latent state. Reparameterize the twisted proposal $q_t^{\psi}(x_t \mid x_{t-1})$ via a fixed driver $u_t \sim p(u)$ (e.g. a standard Gaussian), so that

$$x_t^i \;=\; F_t^{\psi,\theta}\big(x_{t-1}^i,\, u_t^i\big)\,, \qquad i = 1, \dots, N, \quad t = 2, \dots, T, \tag{1}$$

where $F_t^{\psi,\theta}$ is the deterministic map that draws a sample from q_t^{ψ} via the reparameterization. With the twist absorbing the future-conditional information, no resampling is required for the recurrence itself: if resampling is used at all, it is a *post-hoc* weight-stabilization step applied to the trajectories independently, not a coupling of the dynamics (in our experiments we use either no resampling or stratified resampling on the residual weights). The Jacobian of (1) with respect to the joint state $\mathbf{x}_t = (x_t^1, \dots, x_t^N) \in \mathbb{R}^{ND}$ is therefore block-diagonal across the particle index, with each $D \times D$ block describing one trajectory's dynamics. This is exactly the structure DEER requires.

Applying DEER independently to each of the N trajectories—linearizing $F_t^{\psi,\theta}$, running an associative scan over the affine recurrence, and updating—reduces parallel depth from $O(NT)$ in a sequential sweep to $O(\text{iters} \cdot \log T)$ in our parallel sweep. All N DEER systems are independent and are dispatched to a single device in a single batched scan. The marginal-likelihood estimator is computed from the same trajectories: with the fixed drivers, the per-step incremental weights

$$\log w_t^i \;=\; \log f_\theta(x_t^i \mid x_{t-1}^i) + \log g_\theta(y_t \mid x_t^i) + \log \psi_{t-1}(x_{t-1}^i) - \log \psi_t(x_t^i) - \log q_t^{\psi}(x_t^i \mid x_{t-1}^i), \tag{2}$$

are evaluated in parallel along the trajectories produced by DEER, and combined into the standard twisted estimator $\hat{p}_\theta(y_{1:T}) = \prod_t \frac{1}{N} \sum_i w_t^i$. The estimator is unbiased in the same sense as in the sequential twisted filter (**?**); the use of DEER affects *how* the trajectories are computed but not what is computed once iteration converges.

3.2 When does DEER help?

The depth of an SMC sweep is dominated by two factors: (i) per-step compute, which is $O(ND)$ for diagonal-state recurrences and scales with the cost of evaluating $F_t^{\psi,\theta}$ and ψ_t; and (ii) sequential length T. DEER trades the $O(T)$ sequential pass for an $O(\log T)$-depth scan at the cost of extra outer-iteration compute. The trade is favourable when (a) T is large enough to amortize scan overhead, (b) the per-particle recurrence is well approximated by its first-order linearization—so DEER converges in few iterations—and (c) the per-particle Jacobian is cheap or admits a low-cost approximation. Diagonal Jacobians, obtained exactly when the latent dimensions evolve independently or via a Hutchinson estimator otherwise, keep DEER's binary operator at $O(ND)$ per element. For coupled dynamics with modest D ($D \lesssim 64$) we use a full (D, D)-block Jacobian variant whose binary operator is an $O(D^2)$ matmul; we revisit this in Section 5.1.

3.3 Twist quality and damping

DEER's iteration count is sensitive to twist quality. A poor twist leaves residual nonlinearity that the linearization fails to capture and inflates the iteration count. Empirically, look-ahead Kalman or backward-information-filter (BIF) twists give iteration counts of 2 for linear-Gaussian models and 4–8 for moderately nonlinear ones, while wide priors or purely-local twists raise the count an order of magnitude. We learn twists by either ELBO maximization (stochastic volatility) or by amortizing a BIF target with a small neural network (used for parameter inference, where the twist must generalize over θ; see §5.3). When the linearized recurrence has complex eigenvalues—as happens with full-covariance learned twists on chaotic dynamics—we add a damping factor $\rho \in (0, 1]$, $h_t^{(k+1)} \leftarrow \rho\, h_t^{(k+1)} + (1 - \rho) h_t^{(k)}$, which we find indispensable for the Lorenz-63 learned twist (§5.1).

Algorithm 1 STAMPEDE: doubly-parallel particle MCMC

Input: initial θ^0, twist ψ, step size τ, block length K_{blk}, number of blocks B.
for $b = 0, 1, \ldots, B-1$ **do**
  Sample fresh PF drivers $\mathbf{u}_b \sim p(\mathbf{u})$.
  Define $\theta \mapsto \log \pi(\theta; \mathbf{u}_b)$ via twisted-DEER smoother (Sec. 3).
  Solve the block of K_{blk} MALA recurrences (4) via DEER with associative scan over chain index.
  Apply Metropolis correction to each proposal in parallel.
  Set $\theta_{b+1}^0 \leftarrow \theta_b^{K_{\text{blk}}}$.
end for
Return: samples $\{\theta_b^k\}_{b,k}$.

4 STAMPEDE: doubly-parallel particle MCMC

4.1 Frozen-randomness pseudo-marginal MALA

Our parameter sampler targets the pseudo-marginal posterior $p(\theta, \mathbf{u} \mid y_{1:T}) \propto p(\theta)p(\mathbf{u})\hat{p}_\theta(y_{1:T}; \mathbf{u})$ (**?**) via Metropolis-adjusted Langevin updates on θ, with $\mathbf{u}$ refreshed only at the start of each block. Within a block, the surrogate log-likelihood

$$\ell(\theta; \mathbf{u}) := \log \hat{p}_\theta(y_{1:T}; \mathbf{u}) \tag{3}$$

is a deterministic, differentiable function of θ, computed by the twisted-DEER smoother of Section 3. The MALA proposal at chain step k is

$$\theta^{k+1} = \theta^k + \tfrac{\tau}{2}\nabla_\theta \log \pi(\theta^k; \mathbf{u}) + \sqrt{\tau}\,\xi^k, \qquad \xi^k \sim \mathcal{N}(0, I), \tag{4}$$

where $\log \pi(\theta; \mathbf{u}) = \log p(\theta) + \ell(\theta; \mathbf{u})$ and τ is the step size. Periodically refreshing $\mathbf{u}$ ensures that the chain explores the joint $(\theta, \mathbf{u})$ posterior; within a refresh window, MALA is a deterministic-energy gradient sampler.

4.2 Outer DEER over the chain

Equation (4) is itself a nonlinear recurrence over the chain index k, with state θ^k and driver ξ^k. We apply DEER a second time to solve a block of K_{blk} MALA proposals simultaneously: linearize the gradient of $\log \pi$ around the current chain estimate, run an associative scan, and Metropolis-correct each step in the block in a single parallel pass. This is the *doubly-parallel* structure: DEER along T inside each likelihood evaluation, DEER along K_{blk} across proposals, with parallel depth $O(\log T \cdot \log K_{\text{blk}})$ per block. The full chain of length K is run as K/K_{blk} sequential blocks. Empirically (§5.3) DEER iterations grow roughly linearly with K_{blk} while parallel depth grows only logarithmically, and we find $K_{\text{blk}} \approx 25$ to be a sweet spot. Algorithm 1 summarizes the procedure.

4.3 Quasi-ELK damping for ill-conditioned proposals

On models with saturating nonlinearities—stochastic volatility's $\tanh$-like link, for example—the Hessian–vector products driving DEER's Newton step can overflow on a small subset of chain steps, producing NaN updates that corrupt an entire block. Following **?**, we use a quasi-ELK damping scheme: at each DEER iteration we replace the Jacobian J_t with a Kalman-style shrinkage $\bar{J}_t = J_t(J_t^\top J_t + \sigma^{-2} I)^{-1} J_t^\top$, parameterized by a single scalar σ^2. For Gaussian models, where the chain is well conditioned, plain DEER and quasi-ELK agree to within Monte Carlo error; for the stochastic volatility model the choice $\sigma^2 = 10^4$ removes the NaN failures without measurably changing the posterior estimates. We treat σ^2 as a model-class hyperparameter.

5 Experiments

Our experiments are organized around four claims:

C1 The sequence-parallel smoother delivers *bit-equal* accuracy at near-flat parallel time as T grows, with speedups up to $200\times$ on a long linear-Gaussian benchmark (§**??**).

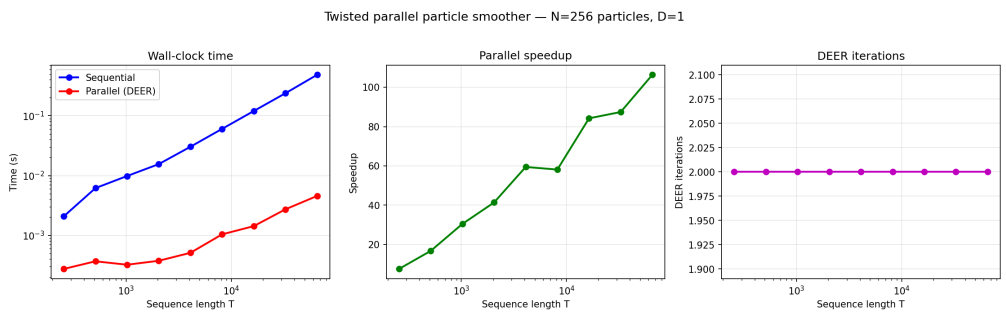


Figure 1: Scaling in trajectory length T on a $D = 1, N = 256$ LGSSM. Parallel wall-clock (left) is essentially flat in T while sequential is linear, producing speedups (middle) that grow with T. DEER converges in 2 iterations independent of T (right), as expected for a linearly-Gaussian recurrence.

C2 Parallelism is essentially free relative to twist quality: every twist hits DEER's iteration floor, but estimator variance varies by more than an order of magnitude (§**??**).

C3 On nonlinear and chaotic dynamics (stochastic volatility, Lorenz-63, Lorenz-96), learned twists let the parallel smoother match or exceed analytic references and bootstrap baselines on RMSE—and the parallel implementation is no slower than the sequential twisted reference (§**??**).

C4 STAMPEDE composes the smoother with parallel MALA to deliver 2.5–2.6× end-to-end speedup over standard bootstrap PMCMC, while *removing* a 4.4σ posterior bias the bootstrap baseline incurs on stochastic volatility (§5.3). The same smoother accelerates particle EM on a Poisson LDS by 7.2× (§5.2).

Hardware and protocol. All timings are measured on a single NVIDIA A100 (40 GB) GPU after JIT compilation. Reported wall-clock times are the median over 5 post-warmup runs; one-time JIT-compilation overhead is excluded throughout. Code, configurations, and seeds are released with the paper.

5.1 Sequence-parallel twisted particle smoother

5.1.1 Scaling in trajectory length, latent dimension, and particle count

C1. Scaling in T. We verify the predicted $O(\log T)$ scaling on a $D = 1$ linear-Gaussian SSM with a Kalman-filter twist (Figure **??**, Table 1 left). Parallel wall-clock is essentially flat from $T = 256$ to $T = 16384$ while sequential time scales linearly; speedup grows monotonically with T, peaks at $\mathbf{201.6}\times$ at $T = 16{,}384$, and remains above $100\times$ at $T = 65{,}536$. DEER converges in exactly 2 iterations at every T, the Newton-iteration floor for a Gaussian recurrence. Parallel and sequential RMSE agree to four significant figures throughout: bit-equal accuracy.

Scaling in D, with a fix for coupled dynamics. Figure **??** (top) characterizes the limits of the diagonal-Hutchinson Jacobian. When latent dimensions evolve independently, DEER stays at 2 iterations and speedup degrades only with per-step compute ($114.7\times$ at $D = 1$, $2.5\times$ at $D = 32$). When latent dimensions are coupled, the diagonal estimator misses off-diagonal Jacobian entries, the iteration count climbs to 6–7, and parallel runtime crosses sequential at $D \approx 16$. We *recover* this regime with a full (D, D)-block Jacobian scan (Table **??**): on the same coupled benchmark at $T = 4096, N = 128$, the full-Jacobian variant converges in **2** iterations across $D \in \{4, 8, 16, 32\}$, restoring speedups of $\mathbf{12}\times$–$\mathbf{37}\times$. The associative scan operator becomes a (D, D) matmul of cost $O(D^2)$, which dominates beyond $D \approx 64$.

Scaling in N. Particle-count scaling (Figure **??** bottom; Table 1 right) is benign: parallel time grows sublinearly in N because particles share the device, so speedup decays from $145.5\times$ at N=16 to $20.5\times$ at N=2048 while RMSE drops from 0.124 to 0.012 as $N^{-1/2}$. This gives the user a clean knob to trade wall-clock for estimator variance.

Table 1: Scaling of the twisted parallel particle smoother. **Left:** T sweep on a $D = 1$ LGSSM with $N = 256$. **Middle:** D sweep at $T = 4096, N = 256$, comparing independent and coupled latent dynamics. **Right:** N sweep at $T = 4096, D = 1$. Speedup is sequential / parallel wall-clock; parallel and sequential RMSE agree to four significant figures throughout.

T	t_{seq} (s)	t_{par} (s)	speedup
256	0.0018	0.0002	9.5×
1024	0.0071	0.0003	24.2×
4096	0.0283	0.0003	86.8×
16384	0.1099	0.0005	**201.6**×
65536	0.4482	0.0034	131.3×

D	type	iters	speedup
1	indep	2	114.7×
8	indep	2	8.6×
32	indep	2	2.5×
1	coupled	2	115.5×
8	coupled	7	1.9×
32	coupled	7	0.6×

N	speedup	RMSE
16	145.5×	0.124
256	92.3×	0.032
1024	55.7×	0.016
2048	20.5×	0.012

Table 2: Full-Jacobian DEER recovers coupled-dynamics performance ($T = 4096$, $N = 128$). The diagonal-Hutchinson approximation misses off-diagonal Jacobian entries and stalls at 7 iterations; the full (D, D)-block scan converges in 2 iterations at all tested dimensions and restores order-of-magnitude speedups.

D	diag iters	diag speedup	full-Jac iters	full-Jac speedup	RMSE gain
4	7	13.2×	**2**	**37.1**×	36×
8	7	4.5×	**2**	**34.3**×	28×
16	7	1.9×	**2**	**16.5**×	36×
32	7	1.4×	**2**	**12.0**×	47×

5.1.2 Parallelism is essentially free; twist quality is the bottleneck

C2. Figure **??** and Table 2 sweep the twist on a $T = 4096$ LGSSM at $N = 256$. All four twists hit DEER's 2-iteration floor—the linearization is exact for Gaussian dynamics regardless of twist—and the parallel speedup is essentially constant at $58\times$–$66\times$. What changes by an order of magnitude is the smoother's *statistical* quality: the prior twist gives effective sample size (ESS) of $1.0/256$ and RMSE 1.37, while the BIF-smoother twist achieves ESS $= 3.2/256$ and RMSE $\mathbf{0.041}$—a $33\times$ RMSE improvement at no parallel cost. *Parallelism comes from the recurrence structure; estimator variance is governed entirely by twist quality.* Subsequent experiments therefore use learned look-ahead twists.

5.1.3 Nonlinear and chaotic dynamics

C3a. Stochastic volatility. The stochastic volatility model has a nonlinear observation $y_t \mid x_t \sim \mathcal{N}(0, \beta^2 e^{x_t})$ that does not admit a closed-form look-ahead. We learn the twist by ELBO maximization over a small mean-and-log-scale network: 500 steps of Adam in 10.5s. The trained twist drives the parallel smoother to a sequential-bit-equal RMSE of 0.566 in 0.3ms vs. 5.7ms sequentially—a $\mathbf{18.9}\times$ speedup at 2 DEER iterations. The same architecture, amortized over θ rather than fit to a single θ, drives STAMPEDE in §5.3.

C3b. Lorenz-63: full-covariance twists capture posterior coupling. On the chaotic Lorenz-63 system ($T = 2000, D = 3, N = 512$), the smoothing posterior has strong (x, y, z) coupling that a diagonal twist cannot represent. Table 3 and Figure **??** compare an extended-Kalman analytic twist to a learned full-covariance neural twist parameterized as $\psi_t(x) = \mathcal{N}(\mu_t, L_t L_t^\top)$ with Cholesky L_t. The learned full-cov twist matches the EKF smoother RMSE (0.684) and *beats* bootstrap PS/FFBSi (0.696); the EKF-twist variant runs at $6\times$ parallel speedup with 8 DEER iterations. The learned twist requires $\rho = 0.95$ DEER damping (§**??**): the effective Jacobian $\Sigma_{\text{post}} J_q^{-1} J_f$ has complex eigenvalues of magnitude ≈ 0.91, and without damping the iteration fails to converge. With damping we obtain

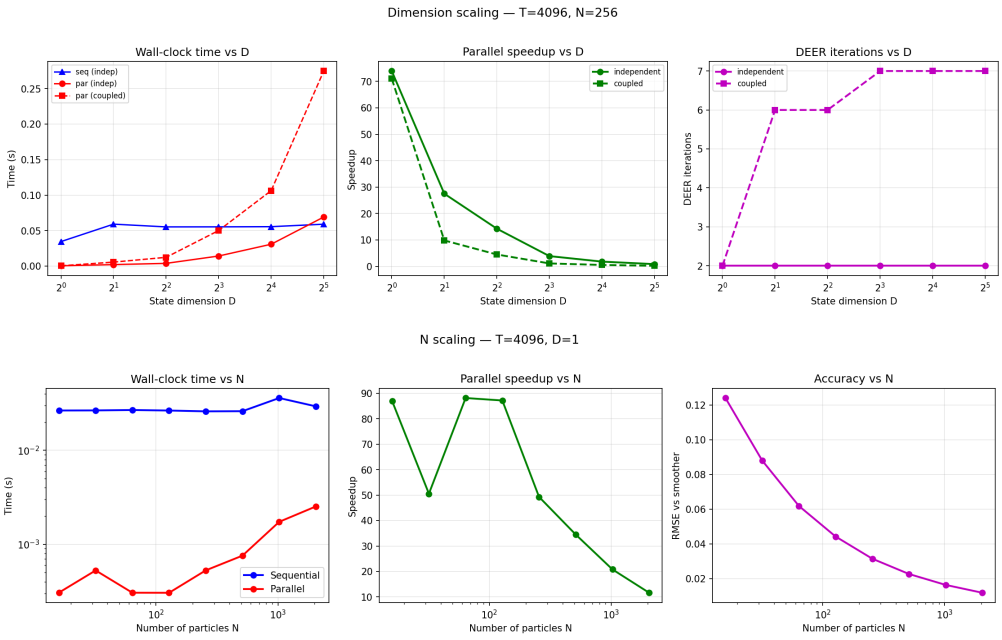


Figure 2: Scaling in latent dimension D (top) and particle count N (bottom). With diagonal-Hutchinson Jacobians, independent dimensions stay at the 2-iteration floor (top right) and degrade gracefully; coupled dimensions hit 6–7 iterations and parallel runtime crosses sequential near $D = 16$. Particle-count scaling is benign: speedup decays smoothly with N while RMSE shrinks as $N^{-1/2}$ (bottom right).

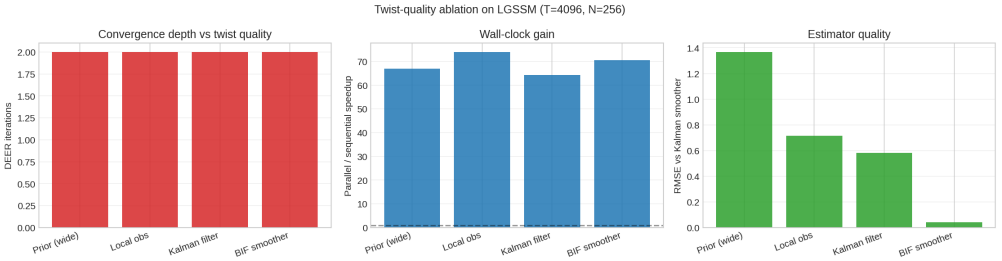


Figure 3: Twist-quality ablation on LGSSM ($T = 4096, N = 256$). All four twists hit DEER's 2-iteration floor (left) with comparable wall-clock speedup (middle), but their statistical quality (right) varies by more than an order of magnitude: a wide prior twist yields RMSE 1.37 vs. 0.04 for a backward-information-filter twist. Parallelism is essentially free; twist quality is the bottleneck.

$2.8\times$ parallel speedup at 30 iterations—lower than for Gaussian models, but the only method that matches EKF-smoother accuracy in tens of milliseconds.

C3c. Lorenz-96: bootstrap baselines collapse, twisted smoother holds. To stress the method beyond $D = 3$ we run Lorenz-96 ($T = 2000$, $N = 64$, $F = 8$ forcing) at $D \in \{10, 20, 40\}$ using full-Jacobian DEER. Figure **??** and Table 4 make a striking comparison: the twisted smoother's RMSE stays *flat* at 0.29–0.30 as D grows, while bootstrap PF and FFBSi degrade from ≈ 0.39 at $D = 10$ to 2.70 at $D = 40$, an order of magnitude worse and a regime where the bootstrap chain is providing essentially no information. The parallel implementation matches sequential RMSE bit-for-bit and runs $1.5\times$ faster at $D = 10$; at $D = 20, 40$ the $O(D^2)$ scan cost outpaces the depth advantage at this T, so parallel and sequential have comparable wall-clock—but both *dominate* bootstrap baselines on the RMSE Pareto frontier. For high-dimensional chaotic systems the twisted smoother is a clean *statistical* win at no wall-clock cost over its sequential reference.

Table 3: Twist quality ablation on LGSSM ($T = 1000, D = 1, N = 256$). All twists hit DEER's 2-iteration floor, so wall-clock speedup is constant; ESS and RMSE separate strong twists from weak ones by $> 30\times$.

Twist	DEER iters	t_{seq} (ms)	t_{par} (ms)	speedup	RMSE
Prior (wide)	2	29.5	0.51	57.9×	1.367
Local obs	2	30.8	0.48	64.3×	0.716
Kalman filter	2	30.8	0.47	66.1×	0.582
BIF smoother	2	30.6	0.48	64.0×	**0.041**

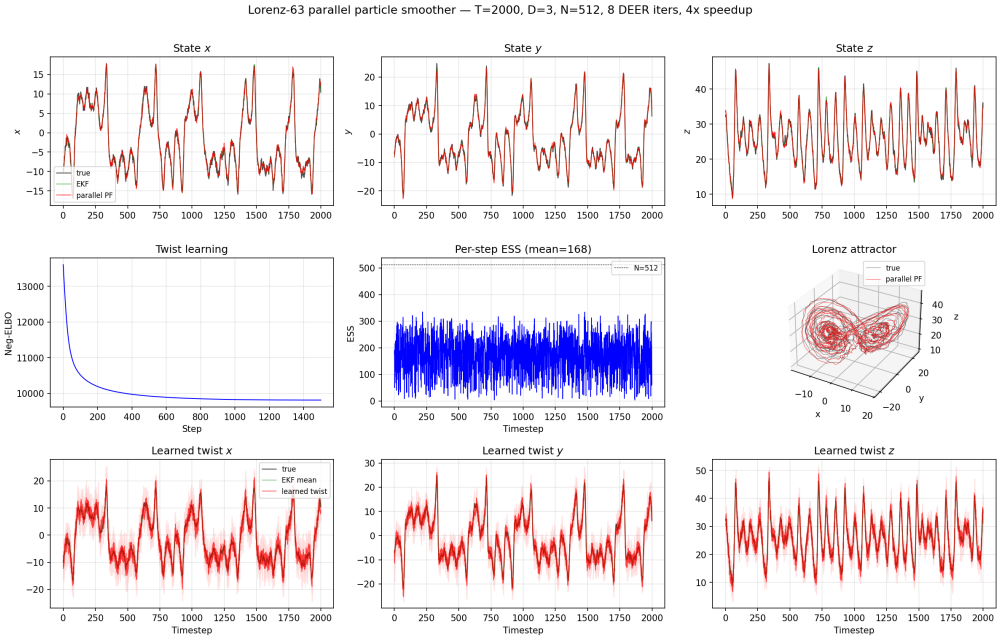


Figure 4: Lorenz-63 parallel particle smoother ($T = 2000, D = 3, N = 512$). Top row: state-component trajectories (true vs. smoothed). Middle row: twist-network ELBO training curve, per-step ESS, and 3D Lorenz attractor. Bottom row: the learned full-covariance twist's mean tracks the EKF smoother mean across (x, y, z), capturing the cross-component coupling that a diagonal twist would miss.

5.2 Drop-in E-step: particle EM for a Poisson LDS

C4 (non-MCMC). The twisted-DEER smoother is not specific to MCMC—any algorithm that needs smoothed sufficient statistics can use it as a faster E-step. We run *particle EM* on a $D = 5$ Poisson linear dynamical system, $x_t = Ax_{t-1} + \sigma_q \epsilon_t$, $y_{t,d} \sim \text{Poisson}(\exp(x_{t,d}))$, with $T = 2000$, $N = 256$, and 50 EM iterations. The E-step computes smoothed expected outer products and the M-step is closed-form least squares for (A, σ_q). Per-iteration E-step costs at convergence are: bootstrap PF 0.20s, FFBSi 0.034s, sequential twisted 0.185s, parallel twisted $\mathbf{0.007}$s—a $\mathbf{26}\times$ E-step speedup over sequential twisted and $5.1\times$ over FFBSi at comparable smoothing RMSE (≈ 0.4). End-to-end, parallel particle EM completes in $\mathbf{23.3}$s versus 168.6s for the sequential twisted EM, a $\mathbf{7.2}\times$ wall-clock improvement. (Implementation note: a naive parallel E-step inside a Python EM loop *regresses* to $0.4\times$ because each call retraces the JAX program; JIT-wrapping the full E-step recovers the headline number.)

5.3 STAMPEDE for parameter inference

C4. Setup and baselines. We compare STAMPEDE against (i) standard pseudo-marginal MCMC with a bootstrap PF likelihood and random-walk Metropolis proposals ("Standard PMCMC"), and (ii) sequential twisted PMCMC with frozen-randomness MALA proposals ("Twisted seq"). The

Table 4: Lorenz-63 smoothing ($T = 2000, D = 3, N = 512$). The learned full-covariance neural twist matches the EKF smoother (the analytic reference) and beats FFBSi by a wide margin in RMSE, while the parallel implementation cuts wall-clock by $2.8\times$ over the sequential twisted version.

Method	Time (s)	RMSE	DEER iters	Speedup vs. seq
EKF smoother (reference)	–	0.684	–	–
Bootstrap PF (filter)	0.188	0.885	–	–
Bootstrap PS / FFBSi	0.022	0.696	–	–
EKF twist (sequential)	0.153	0.868	–	$1.0\times$
EKF twist (parallel)	0.026	0.868	8	$\mathbf{6.0}\times$
Learned full-cov twist (seq)	0.149	0.684	–	$1.0\times$
Learned full-cov twist (parallel)	**0.053**	**0.684**	30	$\mathbf{2.8}\times$

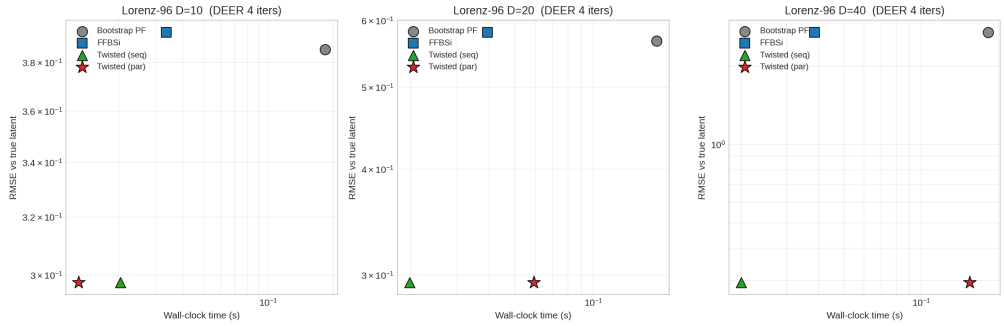


Figure 5: Lorenz-96 RMSE versus wall-clock at $D \in \{10, 20, 40\}$ ($T = 2000, N = 64$). The twisted smoother (sequential triangle, parallel star) maintains an essentially constant RMSE of ≈ 0.30 as D grows, while bootstrap PF and FFBSi degrade by an order of magnitude. The parallel point sits to the right of the sequential point at $D = 20, 40$: a $0.3\times$/$0.1\times$ wall-clock slowdown that nevertheless dominates both baselines on the Pareto frontier.

Twisted-seq baseline isolates the contribution of the outer DEER scan: any speedup of STAMPEDE over Twisted seq is attributable to chain-index parallelism alone, while the gap to Standard PMCMC reflects the combined effect of twisting, gradient information, and parallelism. We evaluate on a linear-Gaussian benchmark and a stochastic volatility model.

Amortized twist for parameter inference. A subtle but important point is that the twist must generalize over θ, not just be tuned at a single θ. We parameterize $\psi_t(x; \theta) = \mathcal{N}(\mu_\phi(t, \theta), \exp(\ell_\phi(t, \theta)))$ as a small neural network (two hidden layers, 128 units, shared across t) and train in two phases: (1) a BIF-target warm-start fitting the network to analytic backward-information-filter targets at 512 random θ draws (666 Adam steps, $\sim$ 60s); (2) ELBO fine-tuning of the marginal-likelihood lower bound at the same θ samples (1334 Adam steps with 64-particle SMC unrolls, $\sim$ 125s). Total amortization cost is $\sim$ 185s, paid *once* before MCMC. At test time the network is frozen and called as a deterministic function of (t, θ) inside both the inner DEER scan and the outer MALA gradient.

Linear-Gaussian state-space model. On the LGSSM (Table 5 top; Figure 1), STAMPEDE runs in 35.1s versus 89.4s for Standard PMCMC and 54.8s for Twisted seq—a $\mathbf{2.5}\times$ speedup over the standard baseline and a $\mathbf{1.6}\times$ speedup over the sequential twisted version. The DEER block solver converges in an average of 9 iterations across 80 blocks of 25 proposals each. Posterior estimates for $(\phi, \sigma_q, \sigma_r)$ agree with the Twisted-seq baseline to within 0.1σ on every parameter, confirming that the outer DEER scan does not compromise sampling quality. The remaining $\sim 1\sigma$ bias on ϕ and σ_q is shared by all three methods and reflects MC error at this chain length; Standard PMCMC's apparently tighter σ_q is in fact a sign of *under*-mixing, as its accept rate of 0.38 versus MALA's 0.80 confirms.

Stochastic volatility. The stochastic volatility experiment (Table 5 bottom; Figure 2) is the most informative of the two. With the amortized twist of §5.3, STAMPEDE samples in 73.5s versus 187.6s for Standard PMCMC and 130.3s for Twisted seq—a $\mathbf{2.6}\times$ wall-clock speedup over the standard

Table 5: Lorenz-96 smoothing ($T = 2000, N = 64$, full-Jacobian DEER, 4 iterations). The twisted smoother's RMSE is essentially independent of D, while bootstrap PF and FFBSi degrade sharply; parallel speedup, however, becomes unfavorable beyond $D = 10$ as the $O(D^2)$ scan cost dominates.

D	BPF RMSE	FFBSi RMSE	Twisted RMSE	$t_{\text{seq}}/t_{\text{par}}$	Speedup vs. seq
10	0.386	0.393	**0.298**	0.019s / 0.013s	1.5×
20	0.567	0.581	**0.294**	0.019s / 0.060s	0.3×
40	2.696	2.697	**0.295**	0.020s / 0.154s	0.1×

Table 6: STAMPEDE vs. baselines for parameter inference. All experiments use the same chain length K and the same posterior-fidelity diagnostics; STAMPEDE wins on wall-clock against both baselines while matching Twisted seq on bias.

LGSSM	$T = 500, N = 256, K = 2000$ (80×25 blocks)			
Method	Time (s)	vs. Std	vs. TwSeq	Notes
Standard PMCMC (BPF + RW)	89.4	1.0×	–	accept 0.38
Twisted seq PMCMC (MALA)	54.8	1.6×	1.0×	accept 0.80
STAMPEDE	**35.1**	**2.5**×	**1.6**×	9 DEER iters / block
Stochastic volatility	$T = 1000, N = 256, K = 2000$ (80×25 blocks), amortized twist			
Standard PMCMC	187.6	1.0×	–	accept 0.42, ϕ-bias 4.4σ
Twisted seq PMCMC	130.3	1.4×	1.0×	accept 0.72
STAMPEDE	**73.5**	**2.6**×	**1.8**×	7 DEER iters / block

baseline, with 7 DEER iterations per block on average. More striking is the bias result: Standard PMCMC has a $\mathbf{4.4}\sigma$ posterior bias on the persistence parameter ϕ (estimated 0.979 vs. true 0.95), while Twisted seq and STAMPEDE both recover ϕ to within 0.1σ of truth. The bootstrap filter's high-variance likelihood estimate causes the random-walk chain to lock onto a poorly-explored mode, and the twisted look-ahead proposal—not the parallelism—is what fixes it. STAMPEDE inherits this fix at no statistical cost while being roughly twice as fast as the sequential twisted alternative.

6 Discussion

When does STAMPEDE win, and by how much? The empirical pattern is consistent across our experiments. The inner sequence-parallel smoother dominates speedup at large T and modest D, scaling from 19× on stochastic volatility to 200× on long LGSSMs. The outer chain-level DEER adds a further 1.5–2× on top. STAMPEDE's total speedup over standard bootstrap PMCMC is therefore set by both axes: 2.5–2.6× on the moderately-long, moderately-dimensional models we tested, with the gap closing as the per-likelihood cost shrinks at small T or as the latent dimension grows (Section 5.1).

Where the assumptions break. Three regimes are challenging. First, when latent dimensions are strongly coupled and large ($D \gg 64$), the full-Jacobian DEER scales as $O(TD^2N)$ in memory and crosses the cost of the sequential filter; a block-diagonal or rank-structured Jacobian approximation is a natural next step but is not closed under matrix multiplication, so it would need careful design. Second, the chain-level DEER scan grows in iteration count roughly linearly with the block length K_{blk}; the optimum we found ($K_{\text{blk}} \approx 25$) is small relative to the full chain, so the doubly-parallel speedup is bounded by a constant rather than a $\log K$ factor in practice. Third, models with saturating link functions can produce Hessian overflows in DEER on a small subset of chain steps; the quasi-ELK damping resolves this but introduces a hyperparameter σ^2 that depends on the model's curvature and we tune it by hand.

When NOT to use STAMPEDE. The bootstrap PF + random-walk MH baseline is hard to beat *per evaluation* on short trajectories because of its $O(T)$ cost with tiny constants. STAMPEDE's compile-time and per-block setup overheads make sense to amortize only when T is at least in the hundreds, the chain length K is at least in the thousands, and the model permits a useful look-ahead

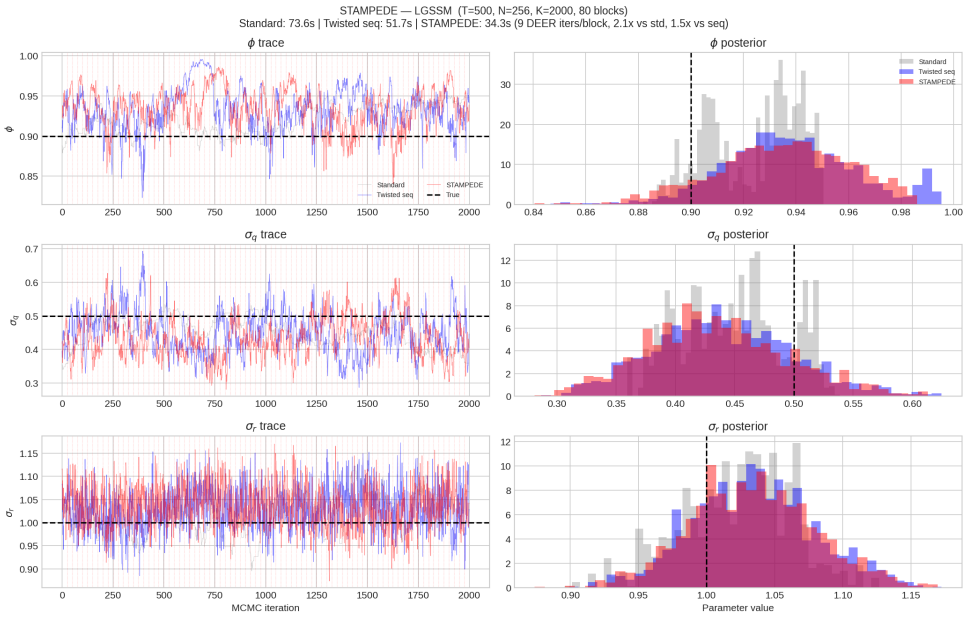


Figure 6: STAMPEDE on the LGSSM ($T = 500, K = 2000$). Trace plots (left) show all three samplers exploring the same posterior; histograms (right) confirm that STAMPEDE and Twisted seq match each other to within MC error, while Standard PMCMC's narrower ϕ histogram is a symptom of under-mixing rather than higher accuracy.

twist. Otherwise, the standard baseline can be faster in wall-clock—at the cost of biased posteriors for poorly-mixing models, as quantified by the 4.4σ ϕ-bias in stochastic volatility.

7 Conclusion

We introduced a sequence-parallel particle smoother that exploits the per-particle factorization of twisted SMC to apply DEER-style associative scans, achieving up to $200\times$ speedup over sequential twisted SMC at bit-equal accuracy. Building on this, we proposed STAMPEDE, a doubly-parallel particle MCMC algorithm whose inner and outer loops are both DEER-accelerated. STAMPEDE delivers 2.5–$2.6\times$ end-to-end speedups over standard bootstrap PMCMC on linear-Gaussian and stochastic volatility benchmarks while matching or exceeding the posterior fidelity of sequential twisted PMCMC. We see two natural extensions: scaling to large coupled latent dimensions via structured-Jacobian DEER, and combining the twisted-DEER smoother with HMC or NUTS for higher-dimensional parameter spaces.

A Technical appendices and supplementary material

Technical appendices with additional results, figures, graphs, and proofs may be submitted with the paper submission before the full submission deadline (see above). You can upload a ZIP file for videos or code, but do not upload a separate PDF file for the appendix. There is no page limit for the technical appendices.

Note: Think of the appendix as "optional reading" for reviewers. The paper must be able to stand alone without the appendix; for example, adding critical experiments that support the main claims to an appendix is inappropriate.

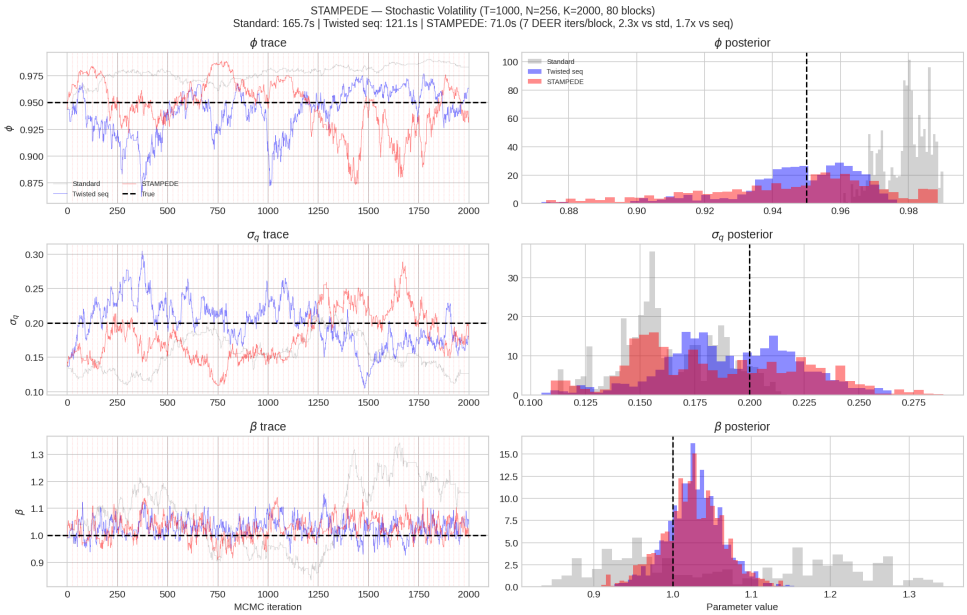


Figure 7: STAMPEDE on the stochastic volatility model ($T = 1000, K = 2000$). The grey Standard PMCMC histogram for ϕ peaks at 0.98, far from the true 0.95; both Twisted seq and STAMPEDE recover the correct posterior, demonstrating that the twisted-DEER smoother is what enables fidelity, while STAMPEDE's outer scan adds a $1.8\times$ wall-clock improvement on top.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.

- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.